

CityEye AI — Smart CCTV Traffic Analytics

Comprehensive Architecture, Multi-Model Vision Pipeline & Technical Specifications

Version: 2.4.0 (Production)	Author / Dev: Shital Salunke	Date: August 17, 2026
Repository: github.com/salunkeshital668-byte/Event-	Framework: YOLOv8 / YOLO11 + FastAPI	Status: Verified & Operational

1. Executive Summary & Project Purpose

CityEye AI is an enterprise-grade, real-time Computer Vision surveillance and CCTV traffic analytics system designed for smart cities, highway authorities, and law enforcement. The platform processes high-density urban traffic video feeds to automatically detect vehicles, track movements, identify traffic rule violations, and immediately flag emergency road hazards and collisions without human intervention.

Key Project Capabilities:

- **Accident & Collision Detection:** Real-time collision flagging using physics IoU overlap and multi-frame deceleration tracking.
- **Helmet Safety Enforcement:** Dual-stage AI detecting motorcycle riders without mandatory safety helmets.
- **Traffic Law Violations:** Detection of triple riding and vehicles driving in the wrong lane/direction.
- **Road Hazard Monitoring:** Instant alerting when vehicles stall or stop unexpectedly in active traffic lanes.
- **Interactive Dashboard & Export:** Web dashboard with live streaming, telemetry counters, and MP4 export.

2. System Architecture & Technology Stack

CityEye utilizes a modern decoupled pipeline combining asynchronous Python web services with high-throughput deep learning inference engines running on PyTorch and Ultralytics.

Component	Technology / Model	Role & Description
Base Object Detection	YOLOv8n / YOLO11n (COCO)	Detects 80 object classes (Cars, Buses, Trucks, Motorcycles, Persons, Traffic lights) at up to 60+ FPS.
Helmet AI Classifier	Custom YOLOv8 Best (helmet_best.pt)	Specialized 2-class model (With Helmet vs Without Helmet) trained for rider head detection.
Multi-Object Tracker	ByteTrack (via Supervision)	Maintains persistent vehicle/person IDs across frames, filtering false positive trajectory jitter.
Backend Web Server	FastAPI + Uvicorn	Serves low-latency multipart/x-mixed-replace MJPEG video streams and REST telemetry APIs.
Video Analytics Engine	OpenCV (cv2) + NumPy	Handles hardware video decoding/encoding, spatial bounding boxes, IoU collision math, and HUD overlays.
Frontend Dashboard	HTML5, Modern CSS, Vanilla JS	Dark-mode glassmorphism interface, real-time metric pills, event log filtering, and full-screen controls.

3. Detailed Event Detection Modules & Physics Logic

CityEye embeds rule-based and spatial-temporal mathematical heuristics layered on top of deep neural network bounding boxes to guarantee zero false-positive accident triggers while capturing critical traffic violations.

Event Name	Category	Detection Logic & Mathematical Criteria	Action / Output
Helmet Violation (helmet_violation)	Safety Law	1. Detects motorcycle and associated riders via spatial overlap. 2. Crops upper head region. 3. Feeds crop to models/helmet_best.pt. 4. Triggers when class is Without Helmet with conf > 35%.	Red HUD box, increments violation counter, logs event with rider ID.

Vehicle Collision / Accident (accident_collision)	Emergency Hazard	1. Computes bounding box IoU overlap between all active vehicle pairs. 2. Checks inter-vehicle centroid distance (< 35 px). 3. Flags when IoU ≥ 0.03 or proximity persists for ≥ 4 consecutive frames.	Logs accident event with both vehicle IDs, IoU score, distance, and timestamp.
Triple Riding (triple_riding)	Traffic Law	Calculates spatial overlap ratio between detected person boxes and the motorcycle box. If ≥ 3 persons overlap the same motorcycle bounding box, triggers violation.	Orange warning banner, increments Triple Riding counter.
Wrong-Way Driving (wrong_way_driving)	Traffic Law	Tracks vehicle center coordinates over a 30-frame temporal history. Calculates displacement vector (Δx, Δy). If movement opposes lane flow (e.g. LEFT when lane is RIGHT) with speed > 1.5 px/frame.	Logs wrong-way incident with vehicle ID and movement vector.
Stopped Vehicle / Hazard (vehicle_stopped)	Traffic Flow	Monitors speed of moving vehicles. If speed falls below 2.0 px/frame for ≥ 1.5 to 4.5 seconds outside predefined signal exclusion zones, flags potential breakdown or obstacle.	Yellow hazard alert, updates Active Hazard counter.
Multi-Class Detection (object_detected)	Telemetry	Continuous frame-by-frame object classification for Cars, Buses, Trucks, Motorcycles, Persons with bounding box coordinates and confidence percentages.	Real-time HUD labels and live telemetry statistics.

4. Multi-Video Processing & Event Data Separation

CityEye dynamically scans the videos/ directory for all supported video formats (.mp4, .avi, .mov, .mkv). It supports both sequential batch execution and individual video selection while strictly guaranteeing that events from different video streams remain isolated.

Video Stream	Input File	Resolution & FPS	Key Events Captured	Output Event Log
Video 1 (Normal Traffic)	videos/input.mp4	480x360 @ 25 FPS (805 frames)	53 NO HELMET violations, 102 vehicle detections, triple riding	data/video_1_events.json data/input_events.json
Video 2 (Accident Scene)	videos/accident.mp4	640x360 @ 20 FPS (340 frames)	11 Vehicle collisions/accidents, 31 stopped vehicle hazards	data/video_2_events.json data/accident_events.json
Combined Archive	All Streams	Multi-resolution	243 total categorized security & traffic events	data/events.json

5. Web Dashboard & Interactive Video Player Features

The CityEye web interface (<http://127.0.0.1:8000>) provides a mission-control operations room experience:

- **Live MJPEG Video Feed:** Streams real-time inference frames with bounding boxes, confidence badges, and HUD overlays.
- **Integrated Play / Pause Controls:** Interactive button (Play / Pause) synchronized with backend streaming.
- **Edge-to-Edge Full Screen:** Seamless full-screen mode utilizing standard browser Fullscreen API with zero black margins.
- **Real-Time Telemetry Counters:** Instantaneous counts for Cars, Buses, Trucks, Motorcycles, Persons, and Safety Violations.
- **Search & Filter Event Log:** Searchable event log filtered by incident type (Helmet, Collision, Wrong-Way, Stopped).

6. Project File Structure & Component Reference

File / Directory	Type	Responsibility & Key Functions
app.py	FastAPI App	Handles /video-feed MJPEG streaming, /live-data telemetry API, /videos list, and static UI routes.
detector.py	AI Detection	Initializes YOLO models, CUDA/CPU thread optimization, and runs frame-by-frame inference with custom confidence.
tracker.py	ByteTrack	Supervision ByteTrack multi-object tracker maintaining consistent track IDs and spatial trajectories.
event_detector.py	Analytics Engine	Core logic for IoU collisions, helmet safety cropping, triple riding, wrong-way detection, and JSON event logging.
main.py	CLI Pipeline	Discovers all videos in videos/, processes sequentially, saves annotated MP4 outputs, and outputs summary.
config.py	Configuration	Centralized path resolution, supported formats (.mp4, .avi, .mov, .mkv), and model constants.
models/helmet_best.pt	YOLO Model	Custom-trained PyTorch weight file for high-accuracy motorcycle rider helmet classification.
templates/index.html	UI Template	Responsive CCTV dashboard structure, video frame container, control buttons, stat cards, and event table.
static/style.css	CSS Stylesheet	Cyber-security dark theme, responsive grid, glassmorphism cards, and edge-to-edge full-screen styling.
static/script.js	Frontend Logic	Asynchronous video stream controller, 200ms telemetry poller, Play/Pause handler, and Fullscreen API listener.

7. Execution Guide & Command Reference

1. Automatic Multi-Video Processing (CLI Batch Runner):

```
python main.py
```

Scans videos/, executes all clips sequentially, outputs annotated MP4s to output/, and creates separate JSON logs.

2. Single Video CLI Processing:

```
python main.py --video videos/accident.mp4
```

3. Launch Web Dashboard (Live Browser GUI):

```
python -m uvicorn app:app --host 127.0.0.1 --port 8000
```

Open <http://127.0.0.1:8000> in any modern web browser to interact with live streaming and detection.